FINITE AUTOMATA AND PUSHDOWN AUTOMATA FOR INPUT VALIDATION AND PARSING

CLA-2 Assessment Tasks 2 and 3 (individual work)

A COURSE ASSIGNMENT REPORT

submitted for the course

21CSC301T – FORMAL LANGUAGE AND AUTOMATA

Submitted by

A.Jagan Karthick (RA2411026050020)

Under the guidance of

Dr. C. GUNASUNDARI

Assistant Professor, Department of AIML

III Year / V Semester / Section A

B.Tech. Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning

SCHOOL OF COMPUTING

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

TIRUCHIRAPPALLI – 621105

NOVEMBER 2026

Task 2: Finite Automata for Real-Time Input Validation

***GitHub**: https://github.com/JAGAN-KARTHICK-A/FLA-Task2-Finite-Automata-Validators*

1. Problem and approach

Registration portals and course systems reject malformed identifiers before using them. I represented three such checks as regular languages: a student roll identifier, a username policy and a course code. Each implementation consumes the input from left to right and records only the finite state needed to decide whether the string follows the selected rule.

The models deliberately separate syntax from meaning. A correctly shaped roll ID does not prove that the student is enrolled, and an allowed course-code pattern does not prove that the course is offered. Those decisions need records maintained by the application.

2. Application 1: Student roll identifier

The language consists of the literal prefix RA followed by exactly thirteen decimal digits. The alphabet is grouped into R, A, D (ASCII digit) and OTHER. q0 reads R, q1 reads A, and q2 through q14 count the thirteen digits. q15 is the only accepting state; qX is a trap state for every invalid or excess character.

```
L_roll = { RAx : x ∈ {0,1,...,9}^13 }
Example: RA2411026050020  -> ACCEPT
```

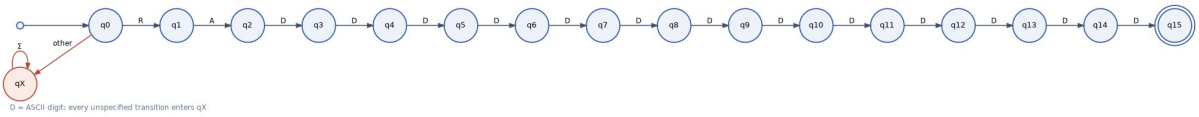


Fig. 1. DFA for the fixed RA prefix and thirteen-digit suffix.

3. Application 2: Username rule

A username must contain between five and fifteen characters. Its first symbol must be an English letter; subsequent symbols may be letters, digits or underscores. States q1 through q15 record the number of consumed characters. q5 through q15 accept, while qX rejects an unsupported character or a sixteenth character.

```
L_user = { w : 5 ≤ |w| ≤ 15, first(w) ∈ L, later(w) ∈ L ∪ D ∪ {_} }
Accepted: Jagan_24     Rejected: 2jagan, name-tag
```



Fig. 2. Length-bounded username DFA; double circles represent valid lengths.

Representative test data (the assignment asks for at least five accepted and five rejected examples per application):

Application	Accepted examples	Rejected examples
Roll ID	RA2411026050020; RA1234567890123; RA0000000000000; RA9999999999999; RA0123456789012	RA241102605002; ra2411026050020; RA24110260500200; RA24110260A020; RB2411026050020
Username	Jagan_24; alpha9; student_name; XyZ_12345; A_bcd; a1234	2jagan; abcd; this_username_is_long; name-tag; name space; _name1
Course code	AIM305; CSE201; ECE110; AIM000; ECE999	AIM30; AIM3000; aim305; AIM3X5; EEE202; CSE20A

Task 2: NFA Construction, Testing and Interpretation

4. Application 3: Course-code validator

The accepted pattern is one of the three uppercase department prefixes AIM, CSE or ECE, followed by exactly three digits. I first represented the union of the prefixes using an epsilon-NFA. Three epsilon branches leave the start state and join the same digit-suffix chain after the selected prefix has been consumed.

```
L_code = {AIM, CSE, ECE}{0,1,...,9}^3
Accepted: AIM305, CSE201, ECE110     Rejected: AIM30, EEE202
```

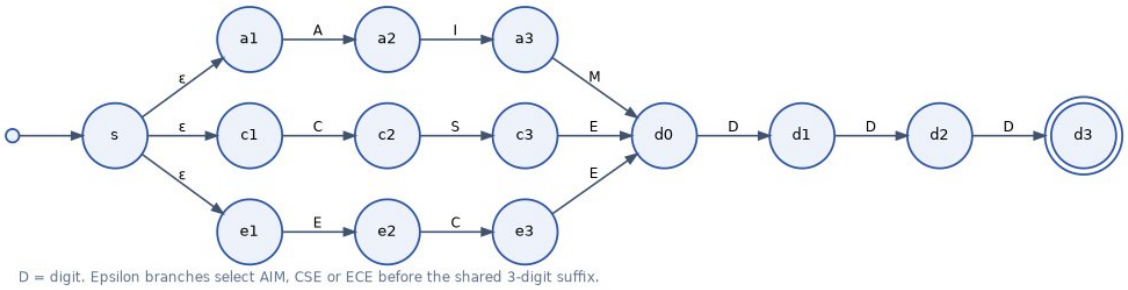


Fig. 3. Epsilon-NFA with three prefix branches and a shared three-digit suffix.

The subset construction forms a deterministic state for each reachable epsilon-closed subset. The generated NFA contains 14 states, the subset DFA contains 12 reachable subsets (including the empty subset), and partition refinement minimises that DFA to 11 states.

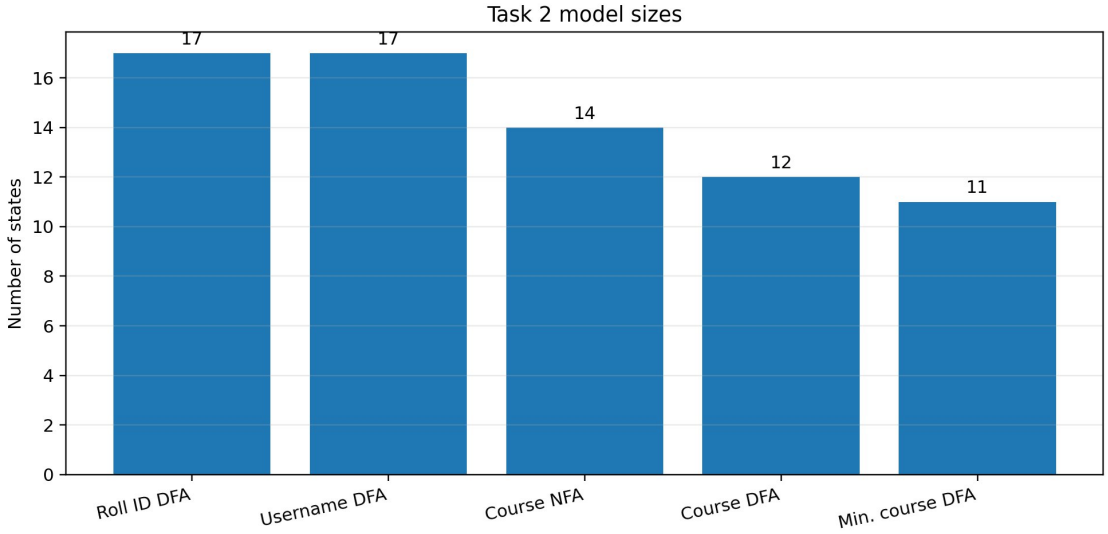


Fig. 4. State counts read directly from the generated machines.

Model	States	Meaning
Roll-ID DFA	17	16 progress states and trap
Username DFA	17	16 length states and trap
Course-code NFA	14	Three prefix paths and shared suffix
Subset DFA / minimal DFA	12 / 11	Reachable subsets; equivalent states merged

5. Validation and observations

Positive and negative examples are exercised in the test suite. The course-code NFA, subset DFA and minimised DFA are compared against one another. An independent regular-expression rule is also used for all 1,111,111 strings of length zero through six over a ten-symbol test alphabet. The tests confirm agreement for this bounded set.

Finite automata are appropriate because the rules need only bounded length information and a finite set of character classes. They do not check whether a username is available, whether a student record exists, or whether a course is open for registration. These are semantic checks performed after syntactic acceptance.

Task 3: Context-Free Grammars and Pushdown Automata

GitHub: *https://github.com/JAGAN-KARTHICK-A/FLA-Task3-PDA-CFG-Case-Studies*

1. Why a stack is required

A finite automaton has a fixed number of states and cannot count an arbitrary nesting depth. Consider L = { an opening parenthesis repeated n times, followed by a closing parenthesis repeated n times : n ≥ 0 }. Let p be a proposed pumping length and choose a string with p opening parentheses followed by p closing parentheses. Since |xy| ≤ p, the non-empty segment y lies entirely among the opening parentheses. Pumping y adds opening parentheses but leaves the closing half unchanged, so the resulting string is not balanced. This contradiction shows that the language is not regular.

A pushdown automaton adds a last-in-first-out stack. The stack stores opening delimiters until matching closing delimiters arrive. The same idea supports recursive arithmetic expressions and nested arrays.

2. Case study A: Balanced delimiters

The target alphabet contains (,), [,], { and }. The grammar is S → (S)S | [S]S | {S}S | ε. It allows empty content, nested groups and concatenated balanced groups. The application includes a small lexical pass that skips characters inside single- and double-quoted strings, so a bracket-like character in a string does not affect the stack.

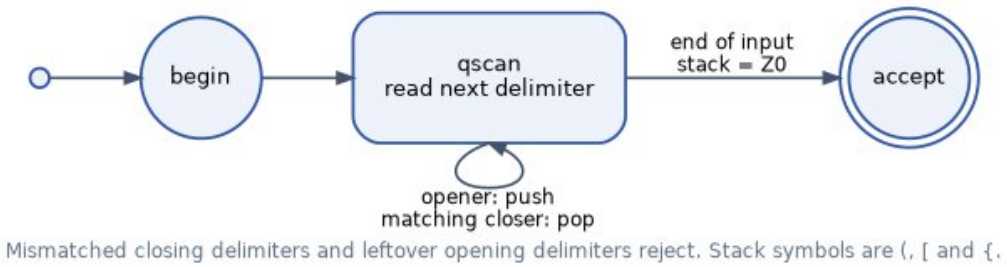


Fig. 5. DPDA control flow. Opening symbols are pushed; matching closing symbols pop the stack.

For the input ({[]}), the stack is initialised with Z0. The top of the stack is shown at the right in the table.

Input	Stack operation	Stack after step
Start	Initialise bottom marker	Z0
(	Push (	Z0 (
{	Push {	Z0 ({
[	Push [	Z0 ({[
]	Match and pop [	Z0 ({
}	Match and pop {	Z0 (
)	Match and pop (	Z0
End	Only bottom marker remains	ACCEPT

The sequence ([)] rejects at the closing parenthesis because the top symbol is [. An unmatched closer or an opening symbol left at end-of-input also causes rejection.

Task 3: Arithmetic Expression Parsing

3. Case study B: Arithmetic expressions

A compact grammar E → E + E | E * E | (E) | NUM accepts basic expressions but does not establish operator precedence. For the token sequence 2 + 3 * 4, one parse groups the addition first and gives (2 + 3) * 4 = 20. Another groups multiplication first and gives 2 + (3 * 4) = 14. The same input therefore has two different parse trees and values under the ambiguous grammar.

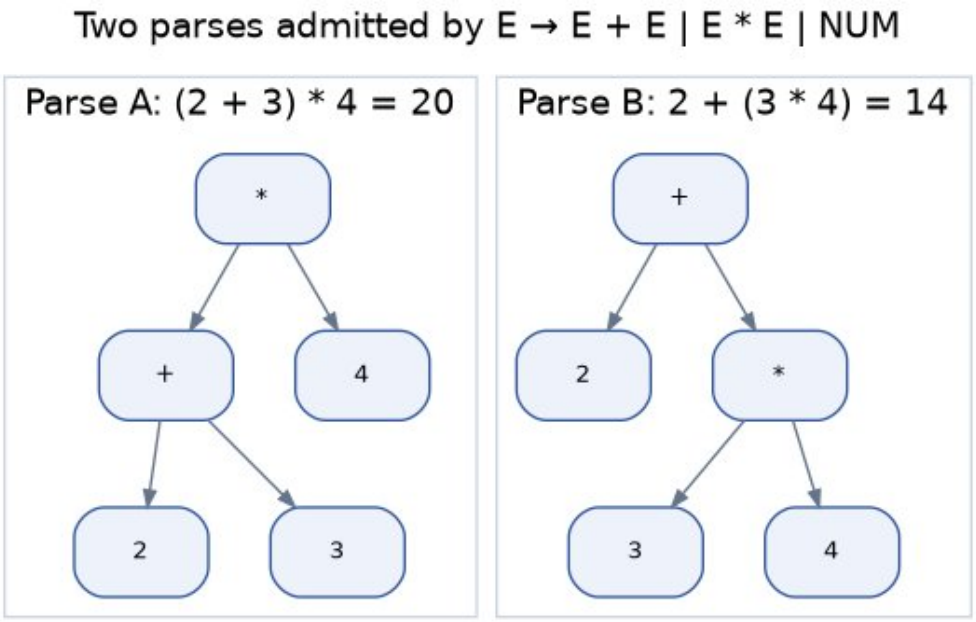


*Fig. 6. Two parse trees for 2 + 3 * 4 under the ambiguous operator grammar.*

4. Precedence-aware grammar

```
E  → T E2
E2 → + T E2 | - T E2 | ε
T  → F T2
T2 → * F T2 | / F T2 | ε
F  → ( E ) | NUM
```

The E level handles addition and subtraction; the T level handles multiplication and division; F handles a number or parenthesised expression. Since multiplication is placed at a lower grammar level, it binds more tightly. The grammar is simulated by the generic CFG-derived NPDA and independently parsed by a recursive-descent evaluator.

Worked input: 2 + 3 * (4 - 1). The parenthesised factor evaluates to 3, multiplication produces 9, and the remaining addition produces 11. The deterministic parser returns the same result as the intended precedence rule.

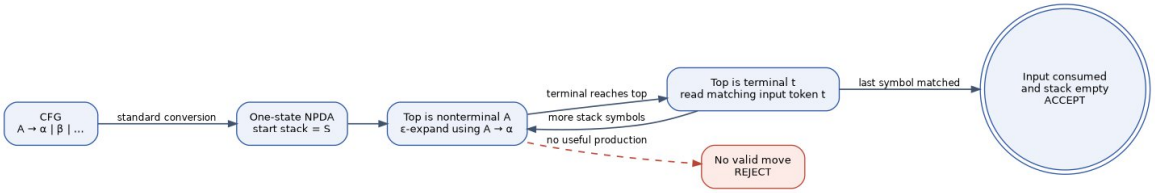


Fig. 7. CFG-to-PDA simulation: expand a nonterminal, match a terminal, and accept on empty stack.

Task 3: Nested Array Syntax and Validation

5. Case study C: Nested arrays

The final case study models a small array-literal language. An array may be empty or contain comma-separated values. A value can be an integer, a quoted string or another array. Recursion is needed because an array may contain another array at any depth.

```
Array    → [ OptItems ]
OptItems → Items | ε
Items    → Value Tail
Tail     → , Value Tail | ε
Value    → NUM | STR | Array
```

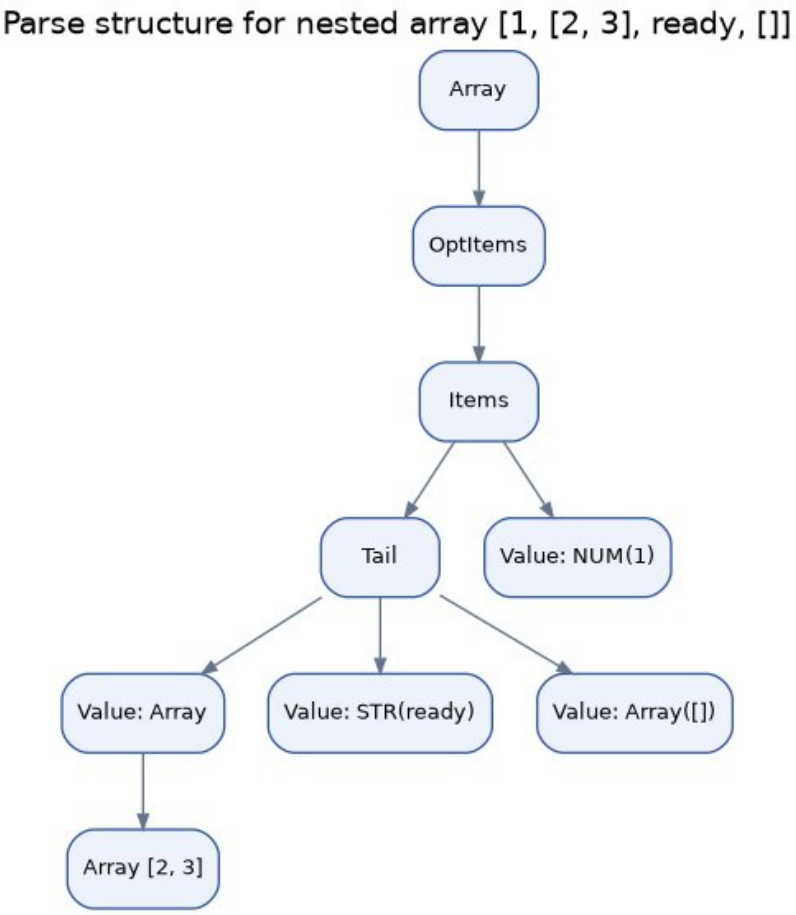


Fig. 8. Parse structure for [1, [2, 3], "ready", []].

For [1, [2, 3], "ready", []], the parser returns a nested Python list containing two integer values in the inner array, a string value, and an empty array. Inputs such as [1,,2], [1 2], [1,] and [1, [2] are rejected because they violate the element or closing-bracket productions.

6. Test cases

Case study	Accepted examples	Rejected examples
Delimiters	(), [], {}, ([{}]), {[()]}	([)], ((),)(, {[}, ()
Expressions	2+3*4, (8-2)/3, 7*5-2, 12, (1+2)*(3+4)	2+, +2, (2+3, 2+*3, 2 3
Arrays	[], [1,2], [1,[2,3],"ready"], ["x"], [7,[8],[]]	[1,,2], [1 2], [1,], [1,[2], 1,2

The explicit delimiter DPDA and CFG-derived NPDA are compared for all 1,555 delimiter strings of length zero through four over the six-symbol bracket alphabet. The expression and array grammars are also checked by the CFG-derived NPDA and their independent recursive-descent implementations.

7. Determinism, Practical Boundaries and Conclusion

Case study	Formal model	Practical parser	Boundary
Delimiters	CFG / PDA	Deterministic stack machine	Full language lexing needs comments and string rules.
Expressions	CFG / CFG-derived NPDA	Recursive-descent parser	No unary operators, names, functions or type rules.
Nested arrays	CFG / CFG-derived NPDA	Recursive-descent parser	Only integers and quoted strings are scalar values.

The grammar-to-PDA construction is nondeterministic in general because a nonterminal may have several productions. In the implementation, breadth-first exploration is bounded at 60,000 configurations to avoid unbounded search. The selected expression and array grammars are right-recursive and small. Separate recursive-descent parsers demonstrate deterministic processing for these chosen rule sets.

The work demonstrates the distinction between regular input checks and nested syntax. Finite automata are suitable for fixed-format identifiers, bounded usernames and course codes. A stack is needed for recursive structure such as nested delimiters, arithmetic parentheses and array literals. The corrected expression grammar resolves the precedence ambiguity shown by the initial grammar.

Individual contribution

This is an individual submission. I defined the languages and grammar productions, designed the automata, implemented the DFA/NFA and CFG/PDA utilities, wrote the validators and parsers, prepared the test cases, ran the verification suite, generated the diagrams and charts, and assembled the report and demonstration interface.

Work area	Contribution
Task 2	Roll-ID and username DFAs; course-code NFA, subset construction and minimisation.
Task 3	Delimiter DPDA; arithmetic and nested-array CFGs; CFG-derived NPDA and parsers.
Verification	Positive/negative and boundary tests; bounded exhaustive equivalence checks.
Documentation	Automata diagrams, parse trees, state-count chart, CLI/browser demo and report.

References

[1] J. E. Hopcroft, R. Motwani and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed., Pearson, 2007.

[2] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2007.

[3] M. Sipser, Introduction to the Theory of Computation, 3rd ed., Cengage Learning, 2012.

[4] SRM Institute of Science and Technology, Tiruchirappalli, Continuous Learning Assessment - 2: Course Assignment Instructions with Rubrics, 21CSC301T - Formal Language and Automata, AY 2026-2027, Tasks 2 and 3.